

E Implementation Details

In this section, we provide a comprehensive overview of the implementation details for our experiments, including the specifics of the baseline methods (Appendix E.1), the full algorithm of our proposed CD-CoT method (Appendix E.2), and the hyper-parameter optimization process (Appendix E.3). For reproduction, the source code and full dialogs are provided at: <https://github.com/tmlr-group/NoisyRationales>.

To conduct the full range of experiments on GPT-3.5-turbo-0613 mentioned in this paper, we utilized a total of **2.03B** tokens. This consisted of 1.21B input tokens and 0.82B output tokens.

E.1 Baseline Methods

In this part, we introduce the implementation details of the baseline methods.

Self-correction Methods:

- **Intrinsic Self-correction (ISC)** [29] asks LLM to endeavor to rectify its initial responses based solely on its inherent capabilities. We employ the prompts from [29], instructing LLMs to review and revise their answers to NoRa tasks with "Review your previous answer and find problems with your answer. Based on the problems you found, improve your answer. Please reiterate your answer."
- **Self-polish (SP)** [89] teaches LLM to eliminate noisy information, rearrange the logic structure, and organize local conditions into new ones in parallel. We implement this method by (1) prompting LLMs to individually refine each noisy CoT exemplar without additional information, repeating the process three times, and (2) combining these rephrased demos to form the context for the task.

Self-consistency Methods:

- **SmoothLLM (SM)** [62] enhances robustness by injecting perturbations into the prompts and utilizing self-consistency to mitigate these effects. We apply the described disturbance methods to noisy rationale demonstrations and feed them into LLMs for reasoning tasks. This process is repeated five times, with the most common answer across iterations selected as the voted answer.
- **Self-denoise (SD)** [102] improves LLM robustness by preprocessing prompts with random masks; the LLMs then work to reconstruct the masked content, reducing noise and aiding incoherent reasoning. Our implementation involves (1) applying the masking method to the noisy rationales on each shot, (2) prompting the LLMs to infer and fill the mask sections of each demonstration, and (3) using the reconstructed CoT demonstrations for task reasoning. This process is also repeated five times, and the most common answer is selected.
- **Self-consistency (SC)** [83] boosts reasoning performance by sampling multiple outputs and conducting majority voting without engaging in any input processing. To apply this method, we run the same task 5 times and vote for the maximum number of the same answers.

External Supervision Methods:

- **Self-correction with Oracle Feedback (SCO)** [29] enhances the ISC framework by providing a definitive ground truth. To implement this method, we use ISC to guide LLMs in refining their reasoning outcomes. The correction process is iterative, allowing for up to two adjustments, but ceases as soon as the correct answer is obtained.
- **Backtracking (BT)** [81] is a method of providing the position where the first mistake occurs to prompt LLMs to correct mistakes. To implement this method, (1) we prompt LLMs to correct the noisy rationales in each CoT example by providing the first noise sentence in it, and (2) we combine these rephrased examples to form the in-context for the task reasoning.
- **ContrastiveCoT (CC)** [9] uses contrastive CoT demonstration to improve LLMs' generalization abilities. We implement this method by providing an extra example with clean rationales as a positive demonstration, and the original prompts with noisy rationales are treated as negative examples. We then prompt the task queries with these positive and negative examples using the contrastive structure prompt mentioned in the paper to guide the LLMs in the reasoning process.

E.2 Contrastive Denoising with Noisy Chain-of-Thought

Step-1: Rephrasing via Supervised Contrasting (1toN). First, we establish a general prompt to construct a pair of contrastive examples, as shown in Sec. 5.1, steering the model towards learning from the clean example and then rephrasing as well as rectifying the noisy examples. It is worth noting that for each dataset, the clean demonstration utilized is fixed, aiming to minimize the usage of additional supervision. In the K -shot in-context learning, we formulate a rephrasing prompt for each noisy example in the context, following the template in Sec. 5.1. The rephrasing time of each noisy example is controlled by the parameter N . In our implementation, we set $N = 5$ to strike a balance between the diversity of rephrased results and computational costs.

Step-2: Rationale Selection (NtoM, $N \geq M$). Given that rephrased results may still contain noise, we employ answer matching for further denoising. Specifically, for each noisy example and its corresponding rephrased rationales, as each rationale can deduce an answer to the question, we select the rationales that the corresponding answers match the given (true) answer of this demonstration. This is called the “answer matching” and does not require an LLM for inference. The selected rationales form the refined candidate pool for that noisy example.

For example, $(x, \mathcal{T}, y)$ indicates the question, rationale, and answer of a given noisy demonstration. The rationale rephrasing (step 1) obtains three rephrased demonstrations: $(x_1, \mathcal{T}_1, y_1)$, $(x_2, \mathcal{T}_2, y_2)$, and $(x_3, \mathcal{T}_3, y_3)$. Then, if $y_1 = y_2 = y$ and $y_3 \neq y$, we will select the first two rephrased rationales, $\mathcal{T}_1$ and $\mathcal{T}_2$. Namely, only rephrased results with consistent answers are retained, forming the refined candidate pool for that noisy demonstration of in-context learning. Notably, in cases where no consistent answer is found, the algorithm randomly selects one of the rephrased results as the candidate pool for the noisy example. Subsequently, we assemble M contexts using the K candidate pools. In particular, we randomly select one candidate from each pool and concatenate the K selected candidates, forming a context. This process is repeated until M contexts are obtained.

Step-3: Rationale Exploration (MtoD, $M \leq D$). For each of the M distinct contexts, we perform multiple reasoning to explore various rationales, resulting in a total number of reasoning times equal to D . To maintain a fair comparison with other baseline methods and to introduce more randomness in generating diverse reasoning paths, we employ the default temperature parameter, *i.e.*, 1. Notably, given the fixed values for M and D , distinct allocation strategies of reasoning times can be generated, leading to performance and cost differences, as illustrated in Tab. 9 and Tab. 10.

Step-4: Answer Voting (Dto1). Given D answers from step 3, we equally vote them into a final answer. For example, if the answer set is 1, 1, 1, 2, 3 where $D = 5$, the answer voting will select “1” as the final answer for its highest frequency.

E.3 Hyper-parameter Optimization

We would like to further clarify the selection of hyper-parameters. To ensure a fair comparison with other baselines (for example, in self-consistency where we have the LLM reason 5 times and then vote), we fixed both D and N to be 5. Specifically, D represents reasoning 5 times over the rephrased results, and N indicates performing contrastive rephrasing on the input sample 5 times. This allows us to study the effects of reasoning with different rephrased inputs.

M represents the number of selected rephrased results from the generated ones, therefore M must be less than or equal to N , and the value of M depends on the breakdown combination of D .

When $D = 5$, the possible combinations are $[(1, 1, 1, 1, 1), (1, 1, 1, 2), (1, 2, 2), (3, 2), (5)]$, with corresponding M s of $[5, 4, 3, 2, 1]$. For example, $(1, 1, 1, 1, 1)$ means inputting 5 different rephrased results and reasoning once for each, totaling 5 times of reasoning. It’s worth mentioning that the greater the value of M , the larger the computational cost.

Finally, to balance the effectiveness and the cost of the algorithm, we test the results on a subset of the dataset with $M = 1, 2, 3, 5$. We find that the accuracy exhibits subtle variations when employing different algorithm instances, with the highest average accuracy observed at 51.3% and the lowest average accuracy at 49.3%. However, the difference in computational cost is relatively significant. Therefore, we ultimately select the algorithm instance with $M = 2$ as the default setting and test it across the entire dataset.

We present the impact of parameters M , D , and C on token usage in Tab. 10. Here, we conduct additional experiments to figure out the effect of varying N on the computational cost. Specifically, we maintain a constant number of reasoning repetitions $D = 5$ while adjusting N and other parameters. Here are the configurations for testing:

1. $N=1$, $M=1$, $C=[5]$, $D=5$
2. $N=2$, $M=2$, $C=[3,2]$, $D=5$
3. $N=3$, $M=2$, $C=[3,2]$, $D=5$
4. $N=4$, $M=2$, $C=[3,2]$, $D=5$
5. $N=5$, $M=2$, $C=[3,2]$, $D=5$ (default)

These experiments are conducted on the NoRa-Math ase-9 task with irrelevant hard noise. The Tab. 30 shows the total number of tokens consumed by CD-CoT for complete reasoning on 300 test samples. This includes tokens used for both rephrasing and reasoning steps.

	$N = 1$	$N = 2$	$N = 3$	$N = 4$	$N = 5$
tokens	1071560	1408845	1532606	1656617	1780095

Table 30: Ablation study of parameter N .

As we can observe, the number of tokens generated increases as N increases. This growth in token count directly correlates with computational needs. However, it’s important to note that the increase is not linear, suggesting that the computational cost does not scale linear proportionally with N .